

Robot Planning Assignment

Martina Panini - Matteo Grisenti

January 2026

1 Introduction

This project addresses the Target Rescue scenario as part of the Robot Planning and its Applications course taught by Professors Palopoli and Focchi. The objective is to develop a robotic planning system capable of rescuing multiple victims within an environment cluttered by obstacles of irregular shapes. As each victim is assigned a specific point value and must be rescued within a defined time limit, the system must implement an efficient strategy to maximize total revenue while ensuring the robot reaches the designated exit gate. To fulfill the course requirements, this project involves the implementation and comparative analysis of both combinatorial and sampling-based planning strategies. The solution is developed in C++ utilizing the `loco_nav` [1] ROS framework.

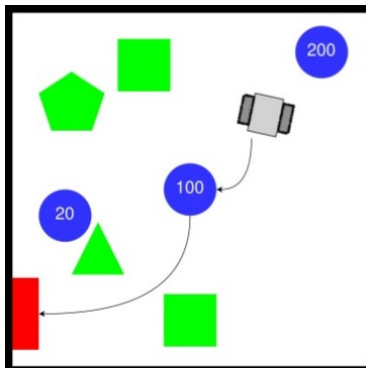


Figure 1: Target Rescue Scenario

2 Planning System Architecture

The objective of this project is to design and implement a hierarchical planning system capable of solving the Target Rescue scenario. The architecture is decomposed into three distinct levels to manage mission logic, topological navigation, and kinematic feasibility.

2.1 Architectural Overview

The system follows a top-down hierarchy:

- **Task Planner (High Level):** Responsible for mission-level logic. It determines the optimal sequence of victims to visit based on rewards and temporal constraints.
- **Graph Planner (Mid Level):** Operates on a roadmap representation of the environment. It employs A^* graph search to find collision-free paths between the target nodes provided by the Task Planner.
- **Dubins Planner (Low Level):** Translates topological nodes into kinematically feasible trajectories, ensuring the robot respects its minimum turning radius R .

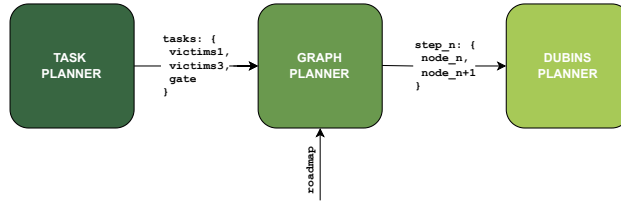


Figure 2: Hierarchical Planner Architecture and Data Flow

2.2 Implementation Strategy: A Decoupled ROS Approach

From a software engineering perspective, the system is divided into two primary ROS packages. This separation is designed to decouple high-level "thought" (logic) from low-level "action" (execution):

1. **robot_planning package:** This contains the **Task Planner** and **Graph Planner**. These are implemented as internal C++ libraries (e.g., `graph_search`) and run within the main mission node. This allows for rapid iteration and decision-making when calculating the global mission sequence.
2. **dubins_planner package:** This is a standalone, decoupled package that provides the **Dubins Planner** via a **ROS Action** (client-server) architecture.

The rationale for using a **ROS Action** interface rather than a simple function call is that the `dubins_planner_server` encapsulates the *execution logic*. By sending a goal to this server, the main mission node can monitor the robot’s progress during long maneuvers and handle real-time feedback or potential interruptions while the server manages high-frequency reference interpolation.

2.3 Task Planner - Greedy Heuristic

To solve the victim selection problem under real-time constraints, the Task Planner implements a **Greedy Heuristic strategy**. Instead of solving an NP-hard global optimization (which would be computationally prohibitive given the real-time constraints), it calculates a utility score $U(v_i)$ for each unvisited victim:

$$U(v_i) = \frac{\text{Value}(v_i)}{T_{\text{travel_to_victim}} + T_{\text{return_to_gate}}} \quad (1)$$

The planner iteratively selects the victim that maximizes this ratio, effectively prioritizing targets that offer a high reward for a relatively low time investment, while always keeping the final exit strategy in consideration.

Time Budget Management

A critical component of the mission logic is the strict enforcement of the time limit. Before adding a candidate victim to the mission sequence, the planner performs a feasibility check to ensure the robot can reach the victim and subsequently navigate to the exit gate within the remaining time.

The validity of a task is determined by the following inequality:

$$T_{\text{elapsed}} + T_{\text{travel_to_victim}} + T_{\text{return_to_gate}} \leq T_{\text{limit}} \quad (2)$$

Conservative Velocity Estimation

A key detail in travel time estimation involves the high-level Task Planner invoking an A* Graph Search (the same used by the Graph Planner) to calculate the summed Euclidean distance between path nodes. Assuming a constant reference velocity, an initial time estimate is derived from this distance.

However, the robot executes motion using Dubins curves, which are inherently longer than straight-line paths due to curvature constraints. To prevent the robot from exceeding its allocated time during execution, we introduced a **conservative velocity** parameter:

$$v_{\text{planning}} = v_{\text{nominal}} \times 0.75 \quad (3)$$

By using this reduced velocity for time estimation ($T = \text{Distance}/v_{\text{planning}}$), the system creates a safety buffer, which aims to compensate for the discrepancy between the topological graph path and the actual kinematic trajectory.

2.4 Graph Planner - Path Smoothing

The raw mission sequence produced by the Task Planner is passed to the Graph Planner. For each mission step, the planner iteratively performs the following:

1. **A* Graph Search:** It invokes an A* search using Euclidean distance as an admissible heuristic, guaranteeing the optimal path relative to the roadmap topology.
2. **Trajectory Refinement:** Since the raw optimal paths often contain redundant waypoints or jagged segments it end up to be unsuitable for kinematic execution. To mitigate this, we apply a Path Pruning post-processing step.

2.4.1 Trajectory Refinement: Path Pruning

Dense roadmap algorithms, such as the Maximum Clearance Roadmap (MCR) and Probabilistic Roadmaps (PRM), tend to generate extremely fragmented paths with negligible waypoint separation (often < 0.2 m). Given the non-holonomic constraints of the Dubins vehicle (turning radius $r = 0.5$ m), these short segments create kinematic bottlenecks. The local planner is often forced to calculate complex maneuvers (e.g., full loops) to align the robot's orientation, resulting in erratic behavior and delays. To resolve this, the pruning algorithm filters the raw trajectory, retaining a node only if:

1. It is a *Critical Node* (Start, Victim, or Gate), ensuring mission objectives are never discarded.
2. Its Euclidean distance from the previously retained node exceeds a safety threshold ($d_{min} = 1.5$ m).

Impact on Performance This refinement transforms jagged paths into smoother Curve-Straight-Curve trajectories, allowing the Lyapunov controller to maintain a constant cruising velocity. For node-dense planners like MCR, this optimization reduced the Total Execution Time (T_{exec}) by approximately 28% (from 107.94s to 77.89s). Importantly, this step is computationally inexpensive, adding a negligible overhead of only 0.002s to the Planning Time (T_{plan}).

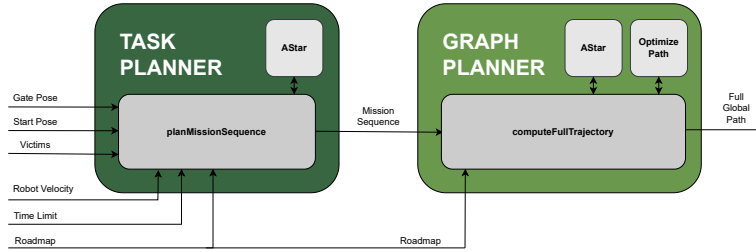


Figure 3: High Level Planner Architecture

2.5 Dubins Planner - Kinematic Execution

The software architecture is centered around a ROS Action Server that manages the lifecycle of a navigation task. When a `DubinsClient` instance sends an action goal containing the target pose, velocity, and turning radius, the `DubinsActionServer` invokes the `DubinsPlanner`. Acting as the high-level coordinator, it sequentially calls:

- **Planning Phase:** The `DubinsSolver` calculates the six classic Dubins primitives (LSL, RSR, LSR, RSL, RLR, LRL) to find the three shortest feasible curve. Then, the `CollisionChecker` sequentially validates the three paths against the environment cache and returns the first valid
- **Execution Phase:** Once validated, the server enters an execution loop (`spin`), interpolating the trajectory and publishing reference poses to the robot's controller at high frequency.

The `DubinsClient` within the main mission node manages this hand-off, sending each node in the smoothed sequence to the server and waiting for completion before requesting the next segment.

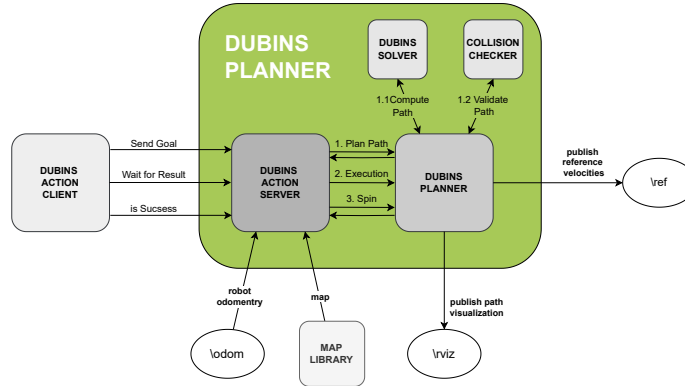


Figure 4: Dubins Planner Architecture and Data Flow

2.6 The Mission Manager: The Central Orchestrator

The **Mission Manager** acts as the executive control center of the planning system. While the previously described architecture organizes the planning process hierarchically, the Mission Manager provides the critical interface that binds these distinct layers together.

It unifies the three architectural levels—Task, Graph, and Dubins—through a sequential, four-phase workflow:

1. **Environment Integration (The Setup)**: Before any planning occurs, the Mission Manager reads the environment map and invokes the roadmap construction process.
2. **Task Planning (High-Level Logic)**: Once the environment is modeled, the Task Planner is called to compute the optimal Mission Sequence based on available targets and time constraints.
3. **Graph Planning (Mid-Level Topology)**: The Mission Manager translates the abstract Mission Sequence into a concrete Navigation Path by invoking the Graph Planner to find collision-free routes between sequence nodes.
4. **Real-Time Execution**: Finally, the Mission Manager shifts from planner to supervisor. It instantiates and manages a Dubins action client to interact with the decoupled Dubins Action Server, monitoring the robot's progress during path execution.

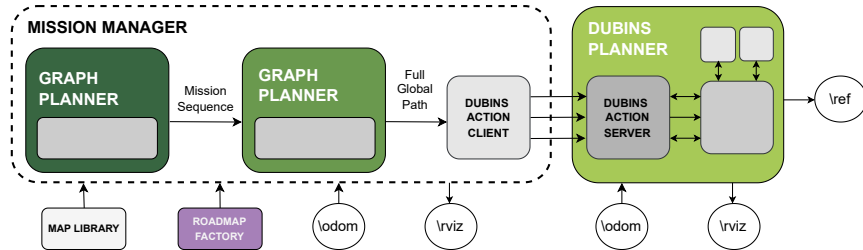


Figure 5: Mission Manager Architecture

3 Roadmap Factory System

The modular architecture of our planning system ensures that the higher-level planners remain independent of the specific environment representation. This is facilitated by a **Roadmap Factory** entity, which generates a standardized roadmap data structure consisting of a set of nodes and edges.

The main difference between the implemented algorithms lies in the strategy used to define these nodes and the connectivity of the edges. By utilizing the Factory pattern, we can easily swap between different spatial partitioning or sampling techniques without modifying the *Graph Planner* logic. The Factory supports the following algorithms:

Combinatorial Planning

- **Exact Cell Decomposition (ECD)**: Implemented using Trapezoidal. The algorithm processes obstacle vertices along the X-axis using a sweep-line approach to generate collision-free trapezoids. The roadmap nodes are placed at the geometric centroid of each resulting trapezoid. Adjacent trapezoids are connected only if the shared vertical boundary exceeds a minimum width (`min_passage_width = 0.70m`), ensuring the robot can physically pass through.
- **Approximate Cell Decomposition (ACD)**: Utilizes a recursive Quadtree-based approach. The configuration space is recursively divided into four rectangular cells until a maximum depth is reached (configured to 4) or cells are fully free. Instead of simply using the geometric center, the algorithm calculates a "Refined Centroid" for each cell. This ensures that the generated node lies strictly within the free space, even if the cell is irregular or partially adjacent to borders.

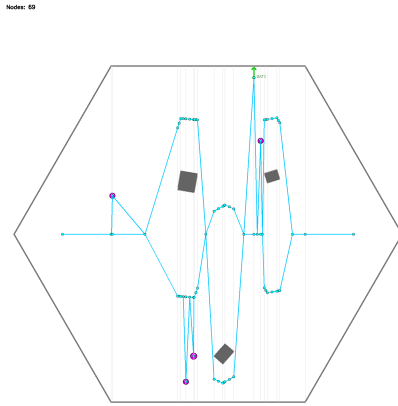


Figure 6: ECD Roadmap

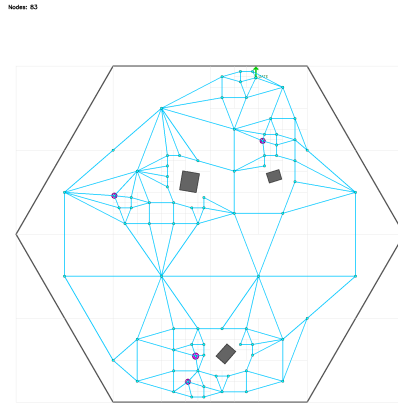


Figure 7: ACD Roadmap

- **Maximum Clearance Roadmap (*MCR*)**: Based on the Generalized Voronoi Diagram (GVD). Unlike standard implementations that use only Voronoi vertices, our approach densely samples the Voronoi edges (every 0.25m) to create smooth paths along the safest routes. Victims and gates are connected to the roadmap by linking them to the $K = 3$ nearest nodes on the Voronoi diagram.
- **Shortest Path Roadmap (*SPR*)**: Implements a Visibility Graph. The roadmap vertices include the Start, Targets, and the reflex vertices (convex corners) of obstacles and map borders. A pre-processing step inflates all obstacles by a padding radius (0.5m). This ensures that "grazing" paths along obstacle edges remain collision-free for the robot's footprint. Edges are added between all pairs of visible nodes.

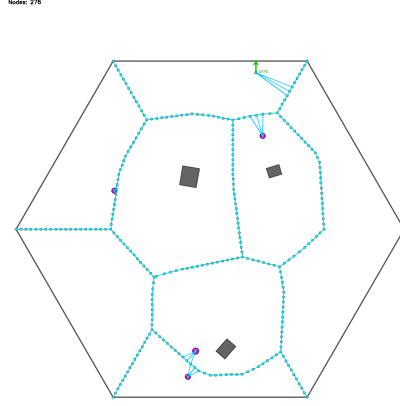


Figure 8: MCR Roadmap

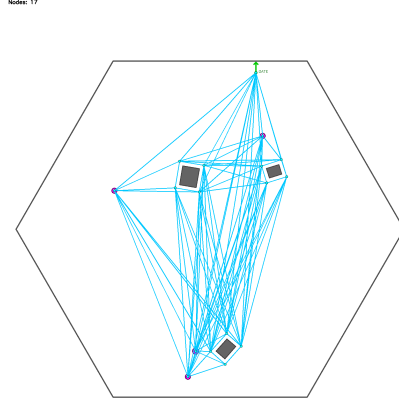


Figure 9: SPR Roadmap

Sampling-Based Planning

- **Probabilistic Roadmap (*PRM*)**: A Multi-Query approach implemented in two phases. Before random sampling, the algorithm explicitly injects critical nodes (Start, Gate, and Victims) to guarantee their presence in the graph. After filling the rest with uniform random samples, nodes are connected to their $K = 15$ nearest neighbors if the path is collision-free.
- **Rapidly-exploring Random Tree (*RRT*)**: A Single-Query tree expansion rooted at the start configuration. The sampling strategy includes a heuristic where, with a probability of 10% (`goal_bias_prob`), the sample is chosen directly from the target set. After the iterations complete, a post-processing step explicitly attempts to connect all targets to the nearest tree node, ensuring exact reachability.
- **Rapidly-exploring Random Tree Star (*RRT**)**: An asymptotically optimal variant of RRT that minimizes path cost. New nodes are not simply linked to the nearest neighbor but to the neighbor within a search radius

($1.2m$) that minimizes the total path cost from the root. The algorithm checks if the new node can provide a cheaper path to existing neighbors. If so, it "rewires" the tree structure locally to improve optimality.

Nodes: 1000

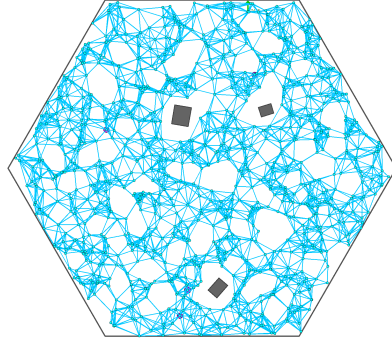


Figure 10: PRM Roadmap

Nodes: 1505

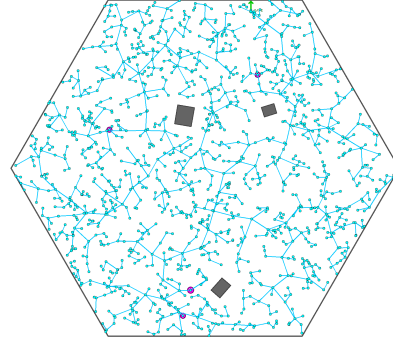


Figure 11: RRT Roadmap

Nodes: 1523

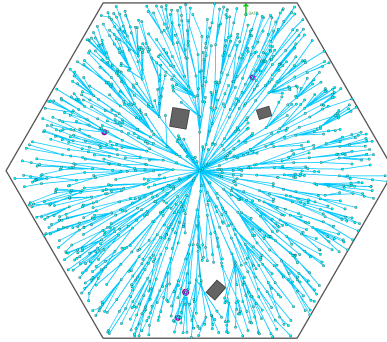


Figure 12: RRT* Roadmap

All implemented planners share a common architecture based on the `Roadmap` class and rely on the `PlanningUtils` library for geometric collision checking.

NB: All the algorithms have been implemented from scratch, with the exception of the **Maximum Clearance Roadmap (MCR)**. The implementation of a robust *MCR* from first principles has been shown to be challenging. Given the project's time constraints and scope, we elected to leverage the `Boost` library, as suggested during the lectures.

4 Secondary System

Along with the previously described planning and roadmap factory, we implemented other elements. Although not directly related to the planning itself, these are necessary components of the system’s workflow.

4.1 Controller

The controller is responsible for bridging the gap between high-level planning and physical execution. As previously mentioned, the execution phase is managed by the `dubins_planner` action server. Once a trajectory is computed, the server begins publishing the reference setpoints; including target position (x, y) , orientation (θ) , and desired linear (v) and angular (ω) velocities, to the desired `/ref` ROS topic.

These references are processed by the controller, which is implemented as a standalone ROS node. The core of this node is a **Lyapunov-based Controller**, designed to minimize the tracking error between the robot’s current pose and the reference trajectory. The controller computes the necessary control laws and publishes the resulting velocities to the `/cmd_vel` topic. These commands are then consumed by the robot’s hardware abstraction layer within the ROS ecosystem.

NB: Although a similar controller is provided within the `loco_nav` framework, we encountered persistent integration issues on our local development environment. Consequently, we opted to reimplement it in the custom ROS package `robot_control`

4.2 The Map Library

The final component of our architectural framework is the `map_library` package. This package implements a centralized library shared across all system components, serving as a data abstraction layer.

Its primary function is to interface with the ROS environment to retrieve raw map data, which is then parsed and stored into organized data structures. Beyond simple data storage, the library provides a suite of essential *utilities*.

The use of a standalone library ensure data consistency across the *Task*, *Graph*, and *Dubins* planners, while significantly reducing code redundancy.

Note on Documentation: For a more comprehensive overview of the project’s software architecture and a detailed breakdown of the directory structure, please refer to the official GitHub repository and its accompanying documentation at: https://github.com/matteogrisenti/robot_planning

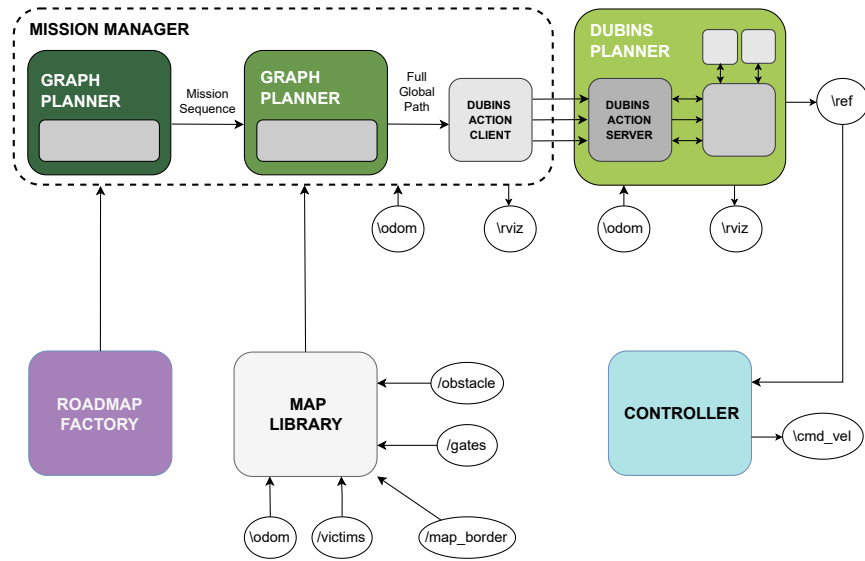


Figure 13: Project Structure

5 Benchmarking Infrastructure

To ensure a rigorous and reproducible validation of the developed algorithms, we implemented a dedicated automated testing framework within the `individual_benchmark_pipeline` ROS node. This infrastructure allows for the systematic execution of the "Target Rescue" scenario under varying constraints without manual intervention.

The pipeline performs the following operations for each test run:

- **Environment Initialization:** Resets the robot pose, obstacles, and victim locations to ensure consistent starting conditions.
- **Automated Metrics Collection:** Records key performance indicators in real-time, including:
 - *Planning Time (T_{plan}):* The cumulative time required for roadmap construction, task planning (victim selection), graph search, and path optimization (smoothing).
 - *Execution Time (T_{exec}):* Total time elapsed from the first motion command to the final stop at the gate.
 - *Total Score:* The cumulative value of all rescued victims.
 - *Dubins Retries:* The number of times the low-level planner failed to generate a feasible trajectory for a graph edge.
- **Result Logging:** Appends the data to a structured `benchmark_results.txt` file for post-processing analysis.

Crucially, the infrastructure enforces the "Time Budget" logic described in Section 2.1.2. If the planner detects that the simulated time exceeds the limit T_{limit} , it forces the robot to return to the gate immediately to secure the collected points.

To ensure reproducibility, all benchmarks were executed with the kinematic and environmental parameters defined in Table 1, enforced directly by the `individual_benchmark_pipeline` node.

Table 1: Simulation and Robot Parameters

Parameter	Value	Description
v_{robot}	0.5 m/s	Nominal linear velocity
R_{min}	0.4 m	Minimum turning radius (Dubins)
$v_{conservative}$	$0.75 \times v_{robot}$	Velocity used for Time Budgeting (85%)
δ_{safety}	0.90 m	Obstacle inflation margin for graph edges
$N_{retries}$	3	Max Dubins connection attempts per edge

6 Experimental Results

We conducted a comparative analysis of both combinatorial (SPR, MCR, ACD, ECD) and sampling-based (PRM, RRT, RRT*) planners. The tests were performed in the "Target Rescue" scenario with three distinct time limits (120s, 60s, and 30s) to evaluate the algorithms' behavior under decreasing temporal resources. Following a tournament-style approach, all algorithms were tested at the maximum time limit, and only the top-performing approaches were selected for stricter time constraints.

The quantitative results are summarized in Table 2.

Table 2: Comparative Benchmark Results

Planner	Limit (s)	Score	Exec T (s)	Plan T (s)	Victims	Retries	Success
<i>Long Duration Scenario (120s)</i>							
ECD	120	1027	68.889	0.005	4	0	YES
SPR	120	1027	74.313	0.002	4	0	YES
PRM	120	1027	76.020	0.023	4	0	YES
ACD	120	1027	76.989	0.004	4	0	YES
MCR	120	1027	77.889	0.008	4	0	YES
RRT*	120	1027	81.223	0.009	4	0	YES
RRT	120	1027	90.086	0.060	4	0	YES
<i>Medium Duration Scenario (60s)</i>							
PRM	60	1003	38.962	0.042	3	0	YES
SPR	60	1003	40.250	0.005	3	0	YES
MCR	60	1003	41.317	0.016	3	0	YES
ACD	60	1003	43.801	0.009	3	0	YES
ECD	60	573	41.556	0.013	2	0	YES
<i>Short Duration Scenario (30s)</i>							
SPR	30	430	29.201	0.005	1	0	YES
PRM	30	430	29.605	0.006	1	0	YES

6.1 Performance Analysis

6.1.1 Convergence in Relaxed Constraints (120s)

With a generous time budget of 120 seconds, all implemented planners converged to the maximum possible score (1027 points, 4 victims) with zero Dubins retries, demonstrating the robustness of the updated execution logic.

- **Impact of Path Pruning on MCR:** A crucial observation is the performance of the Maximum Clearance Roadmap (MCR). In preliminary tests without optimization, MCR suffered from excessive execution times ($> 107s$) due to dense node placement. With the introduction of the *Path Pruning* algorithm, MCR's execution time dropped to 77.889s, making

it competitive with SPR and ACD while maintaining its safety clearance features.

- **ECD Efficiency:** The Exact Cell Decomposition (ECD) achieved the fastest execution time (68.8s). This suggests that for this specific environment, the median paths generated by ECD required fewer steering corrections compared to the visibility graph edges of SPR.
- **RRT Lag:** As expected, the basic RRT planner remained the slowest (90.0s) due to the jagged nature of its random tree, which is suboptimal for non-holonomic robots despite the constant velocity controller.

6.1.2 Degradation under Pressure (60s & 30s)

The differences between planners became critical as the time limit decreased.

- **High Performers (60s):** At 60 seconds, SPR, MCR, ACD, and PRM all successfully rescued 3 victims (Score: 1003). PRM proved to be particularly efficient, recording the lowest execution time (38.9s) in this batch, slightly outperforming the deterministic SPR.
- **ECD Drop-off:** While ECD was the fastest at 120s, it failed to maintain the high score at 60s (Score: 573, 2 victims). This indicates that while ECD paths are smooth to execute, they are geometrically longer than SPR’s visibility graph. In tight time budgets, this extra path length causes the Task Planner to prune the third victim from the sequence.
- **Critical Scenarios (30s):** In the most restrictive scenario, only SPR and PRM were tested. Both demonstrated equivalent robustness, rescuing 1 victim (Score: 430) with nearly identical execution times ($\approx 29s$). The sampling-based nature of PRM did not hinder its performance here, suggesting that with a sufficient number of samples, it approximates the optimal topology well enough for this environment.

6.2 Conclusion on Planner Selection

Results highlight that **SPR** is the most consistent choice for time-critical missions due to its guarantee of finding the shortest geometrical path, which maximizes the budget for victim collection. However, the optimizations applied (Path Pruning and Constant Velocity) have made **MCR** and **PRM** highly competitive alternatives. Specifically, PRM showed efficiency in medium constraints, making it a viable probabilistic alternative to the deterministic SPR.

A Appendix: Path Visualizations

This appendix illustrates the performance of the planners under different time constraints (1200s, 60s, 30s). These visual results highlight how the path quality evolve as the available computation time increases.

A.1 Combinatorial Planners

Shortest Path Roadmap (SPR)

Nodes: 18

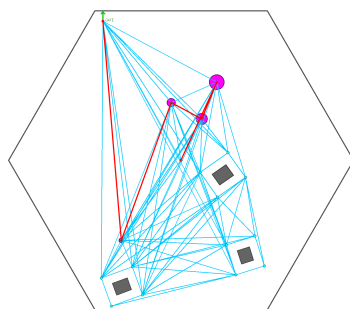


Figure 14: 120 Seconds

Nodes: 18

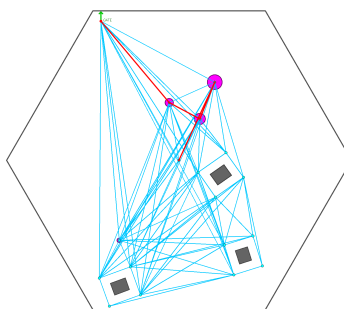


Figure 15: 60 Seconds

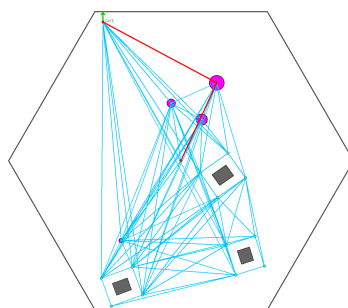


Figure 16: 30 Seconds

Approximate Cell Decomposition (ACD)

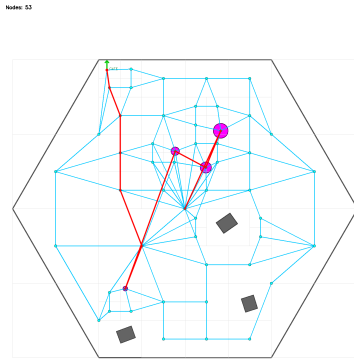


Figure 17: 120 Seconds

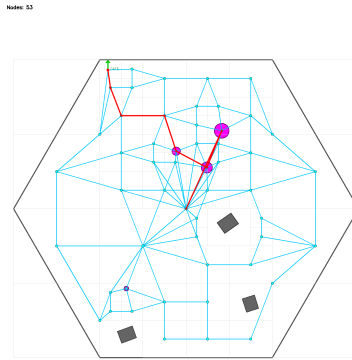


Figure 18: 60 Seconds

Exact Cell Decomposition (ECD)

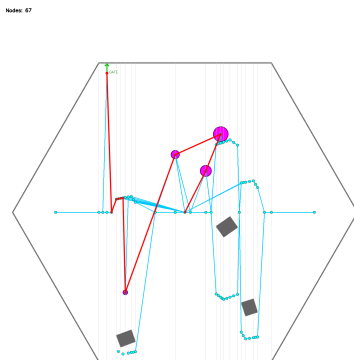


Figure 19: 120 Seconds

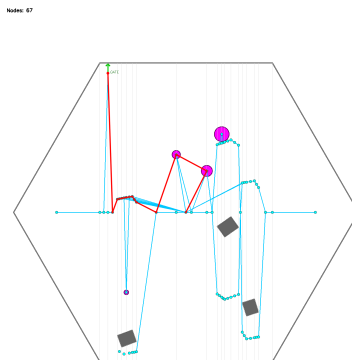


Figure 20: 60 Seconds

Maximum Clearance Roadmap (MCR)

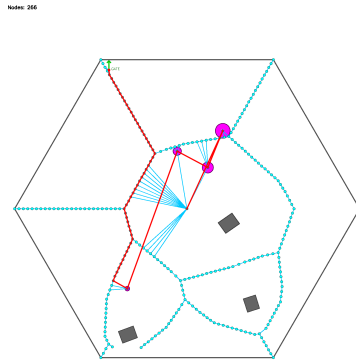


Figure 21: 120 Seconds

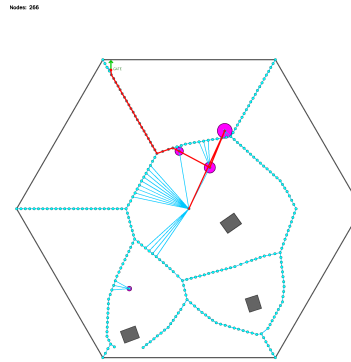


Figure 22: 60 Seconds

A.2 Sampling-Based Planners

Rapidly-Exploring Random Tree (RRT, RRT*)

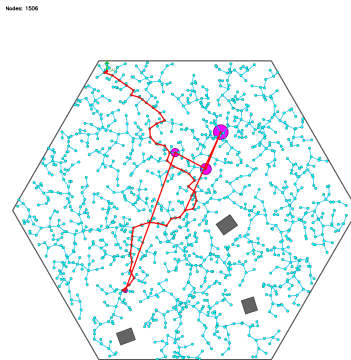


Figure 23: RRT (120s)

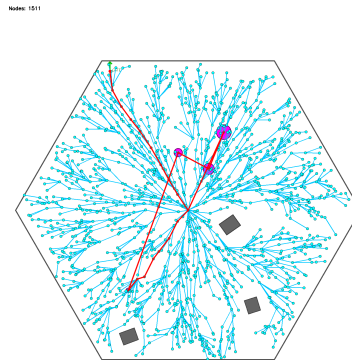


Figure 24: RRT* (120s)

Probabilistic Roadmap (PRM)

Nodes: 1500

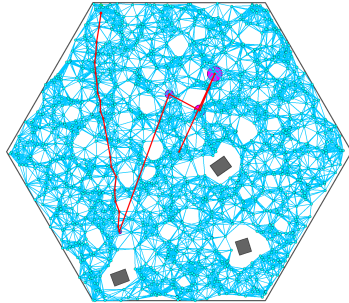


Figure 25: 120 Seconds

Nodes: 1500

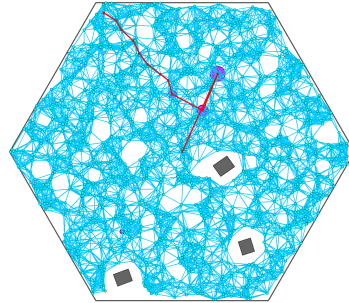


Figure 26: 60 Seconds

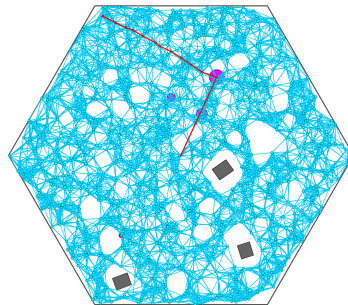


Figure 27: 30 Seconds

References

- [1] Michele Focchi and Luigi Palopoli. loco_nav: A ROS-based framework for robot locomotion and navigation. https://github.com/idra-lab/loco_nav, 2024. Accessed: 2025-01-03.

NB: Large Language Models (LLMs) were utilized as assistive tools for both the code implementation and the technical drafting of this report.